

Week 8 AI Laboratory: Bayesian Networks and Autoregressive Language Models

Advay Matapurkar | ID: 2023A7PS0459G

CS F407 - lab record. Code: bn_lm.py; full output: run_output.txt (seed 407 for the 20-sample sections; seed 1 for the five Part X sampling-mode sentences). All numbers below are copied from that run.

1. Implementation summary

bn_lm.py defines one class NGramLM(order). With order=1 it is the first-order model $P(X_t|X_{t-1})$; with order=2 it is the second-order model $P(X_t|X_{t-2}, X_{t-1})$. Sentences are lower-cased, split into word tokens, padded with <START> (order copies) and one <END>. Transition counts are stored in self.counts[context][token]; the CPT is $\text{self.probs}[\text{context}][\text{token}] = C(\text{ctx}, w) / \sum_k C(\text{ctx}, k)$. predict() is arg max (ties alphabetical), sample() draws from the distribution, and generate(mode) supports greedy and sampling. Only the standard library (random, collections) is used.

2. Conditional probability tables

First-order (selected contexts)

```
P(next | <START>) = {the: 1.000}
P(next | the) = {cat: 0.250, dog: 0.250, mat: 0.167, park: 0.167, rug: 0.167}
P(next | cat) = {sat: 0.667, ran: 0.333}
P(next | dog) = {sat: 0.667, ran: 0.333}
P(next | sat) = {on: 1.000}
P(next | ran) = {to: 1.000}
P(next | on) = {the: 1.000}
P(next | to) = {the: 1.000}
P(next | mat) = {<END>: 1.000}
P(next | rug) = {<END>: 1.000}
P(next | park) = {<END>: 1.000}
```

Zero-probability transitions for the requested words (next in vocab + END):

```
after 'the': <END>, on, ran, sat, the, to
after 'cat': <END>, cat, dog, mat, on, park, rug, the, to
after 'dog': <END>, cat, dog, mat, on, park, rug, the, to
after 'sat': <END>, cat, dog, mat, park, ran, rug, sat, the, to
after 'ran': <END>, cat, dog, mat, on, park, ran, rug, sat, the
```

Second-order (selected contexts)

```
P(next | <START> <START>) = {the: 1.000}
P(next | <START> the) = {cat: 0.500, dog: 0.500}
P(next | the cat) = {sat: 0.667, ran: 0.333}
P(next | the dog) = {sat: 0.667, ran: 0.333}
P(next | cat sat) = {on: 1.000}
P(next | dog ran) = {to: 1.000}
P(next | on the) = {mat: 0.500, rug: 0.500}
P(next | to the) = {park: 1.000}
Unseen context example:
P(next | cat park) = <no observed transitions>
```

Next-word prediction: distribution and arg max, first-order (Part VIII)

```
P(next | the) = {cat: 0.250, dog: 0.250, mat: 0.167, park: 0.167, rug: 0.167}
argmax -> cat
P(next | cat) = {sat: 0.667, ran: 0.333}
argmax -> sat
P(next | dog) = {sat: 0.667, ran: 0.333}
argmax -> sat
P(next | sat) = {on: 1.000}
argmax -> on
P(next | ran) = {to: 1.000}
argmax -> to
```

```

P(next | on) = {the: 1.000}
argmax -> the
P(next | to) = {the: 1.000}
argmax -> the

```

Next-word prediction: distribution and arg max, second-order

```

P(next | the cat) = {sat: 0.667, ran: 0.333}
argmax -> sat
P(next | the dog) = {sat: 0.667, ran: 0.333}
argmax -> sat
P(next | sat on) = {the: 1.000}
argmax -> the
P(next | on the) = {mat: 0.500, rug: 0.500}
argmax -> mat
P(next | ran to) = {the: 1.000}
argmax -> the
P(next | to the) = {park: 1.000}
argmax -> park

```

3. Probability-normalisation tests

For every context w the program checks $\sum_v P(v|w) = 1$ (tolerance $1e-9$). Result: 11/11 first-order and 15/15 second-order contexts sum to 1.000000, 0 failures. As a negative control I subtracted 0.13 from $P(\text{cat}|\text{the})$; the test then reports the 0.87, showing that the test can actually fail.

```

first-order:
<START>          1.000000
cat              1.000000
dog              1.000000
mat              1.000000
on               1.000000
park             1.000000
ran              1.000000
rug              1.000000
sat              1.000000
the              1.000000
to               1.000000
-> 11 contexts checked, failures: 0
second-order:
<START> <START>   1.000000
<START> the       1.000000
cat ran           1.000000
cat sat           1.000000
dog ran           1.000000
dog sat           1.000000
on the            1.000000
ran to            1.000000
sat on            1.000000
the cat           1.000000
the dog           1.000000
the mat           1.000000
the park          1.000000
the rug           1.000000
to the            1.000000
-> 15 contexts checked, failures: 0
Negative control (subtract 0.13 from P(cat|the)): [('the', 0.87)]

```

Unseen-context test

Prediction from an unseen word returns None, and generation continued from an unseen context ('banana' for first-order, 'cat park' for second-order) stops with status UNSEEN in both modes; the script asserts this.

```

m1.predict(('park',)) -> <END> (park is only followed by END)
m1.predict(('banana',)) -> None
first-order generate('greedy') continuing from 'banana' -> tokens ['banana'], status UNSEEN
first-order generate('sample') continuing from 'banana' -> tokens ['banana'], status UNSEEN
second-order generate('greedy') continuing from 'cat park' -> tokens ['cat', 'park'], status UNSEEN
second-order generate('sample') continuing from 'cat park' -> tokens ['cat', 'park'], status UNSEEN

```

4. Generated text

20 sampled sentences, first-order (seed 407)

1. the cat sat on the park [END]
2. the cat sat on the rug [END]
3. the cat ran to the dog sat on the park [END]
4. the dog sat on the rug [END]
5. the dog ran to the cat ran to the dog ran to the rug [END]
6. the cat sat on the rug [END]
7. the cat sat on the dog sat on the mat [END]
8. the cat ran to the mat [END]
9. the cat sat on the dog sat on the dog ran to the cat ran to the rug [END]
10. the cat sat on the mat [END]
11. the park [END]
12. the rug [END]
13. the cat ran to the cat sat on the mat [END]
14. the mat [END]
15. the rug [END]
16. the rug [END]
17. the dog sat on the dog ran to the park [END]
18. the park [END]
19. the mat [END]
20. the rug [END]

distinct: 14/20

20 sampled sentences, second-order (seed 407)

1. the cat sat on the rug [END]
2. the cat sat on the rug [END]
3. the cat ran to the park [END]
4. the dog sat on the mat [END]
5. the dog sat on the mat [END]
6. the dog ran to the park [END]
7. the cat sat on the mat [END]
8. the cat sat on the mat [END]
9. the cat sat on the rug [END]
10. the dog ran to the park [END]
11. the cat sat on the mat [END]
12. the dog ran to the park [END]
13. the cat ran to the park [END]
14. the cat sat on the rug [END]
15. the dog sat on the rug [END]
16. the cat sat on the mat [END]
17. the dog sat on the mat [END]
18. the cat sat on the mat [END]
19. the cat ran to the park [END]
20. the cat ran to the park [END]

distinct: 6/20

Greedy vs sampling

first-order Mode A greedy 1: the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on [MAXLEN]

first-order Mode A greedy 2: the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on [MAXLEN]

first-order Mode A greedy 3: the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on [MAXLEN]

first-order Mode A greedy 4: the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on [MAXLEN]

first-order Mode A greedy 5: the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on [MAXLEN]

distinct in 5 greedy runs: 1

first-order Mode B sample 1: the rug [END]

first-order Mode B sample 2: the dog sat on the cat ran to the park [END]

first-order Mode B sample 3: the park [END]

first-order Mode B sample 4: the rug [END]

first-order Mode B sample 5: the mat [END]

distinct in 5 samples: 4

second-order Mode A greedy 1: the cat sat on the mat [END]

second-order Mode A greedy 2: the cat sat on the mat [END]

second-order Mode A greedy 3: the cat sat on the mat [END]

second-order Mode A greedy 4: the cat sat on the mat [END]

second-order Mode A greedy 5: the cat sat on the mat [END]

distinct in 5 greedy runs: 1

second-order Mode B sample 1: the dog sat on the mat [END]

second-order Mode B sample 2: the cat ran to the park [END]

second-order Mode B sample 3: the dog ran to the park [END]

second-order Mode B sample 4: the dog sat on the mat [END]

second-order Mode B sample 5: the cat ran to the park [END]
distinct in 5 samples: 3

Model comparison

	first-order	second-order
Observed contexts (CPT rows with data)	11	15
Non-zero parameters (context,next) pairs	17	19
Rows in full CPT over vocabulary	11	121
Unobserved contexts (rows with no data)	0	106
Zero entries inside observed rows	104	146
Distinct sentences in 20 samples	14	6
Samples identical to a training sentence	4	20

Novel (not in training data) first-order: ['the cat ran to the cat sat on the mat', 'the cat ran to the dog sat on the park', 'the cat ran to the mat', 'the cat sat on the dog sat on the dog ran to the cat ran to the rug', 'the cat sat on the dog sat on the mat', 'the cat sat on the park', 'the dog ran to the cat ran to the dog ran to the rug', 'the dog sat on the dog ran to the park', 'the mat', 'the park', 'the rug']

Novel (not in training data) second-order: []

Note: the full-CPT row counts include <START> as a context word; the number of rows that can actually be reached is smaller.

5. Answers to Questions 1-14

Q1

Generating a whole sentence directly would need a distribution over all possible sequences. The chain rule breaks that into a sequence of small problems: at each step we only need a distribution over the *next* token given what has been produced so far. Generation is then: sample X_1 , append it, sample X_2 given it, and so on until <END>. Any joint distribution can be written this way, so the decomposition loses no generality.

Q2

Each word depends only on its immediate predecessor: $P(X_t | X_1, \dots, X_{t-1}) = P(X_t | X_{t-1})$. Equivalently, X_t is conditionally independent of $X_1, \dots, X_{t-2}$ given X_{t-1} (the Markov property).

Q3

See the first-order table in section 2. In this dataset 'the' occurs 12 times before another token: cat 3, dog 3, mat 2, rug 2, park 2. So $P(\text{cat}|\text{the}) = P(\text{dog}|\text{the}) = 0.250$ and $P(\text{mat}|\text{the}) = P(\text{rug}|\text{the}) = P(\text{park}|\text{the}) = 0.167$; the 3/5 and 2/5 in the handout are only an illustration. $P(\text{sat}|\text{cat}) = 0.667$, $P(\text{ran}|\text{cat}) = 0.333$, the same for dog; $P(\text{on}|\text{sat}) = 1$; $P(\text{to}|\text{ran}) = 1$. Zero-probability transitions: after 'the': <END>, on, ran, sat, the, to; after 'cat' and 'dog': everything except sat and ran (e.g. cat->mat, dog->park); after 'sat': everything except 'on'; after 'ran': everything except 'to'. The complete lists are in the output above.

Q4

In `NGramLM.counts`, a dict of dicts keyed by context tuple and then by next token, filled inside `fit()`.

Q5

At the end of `fit()`, where each row of counts is divided by its row total and stored in `self.probs`.

Q6

`generate('sample')` samples from $P(w|\text{prev})$ using `random.choices` with the probabilities as weights. `generate('greedy')` always takes the arg max. Greedy is deterministic and follows the single most likely word at each step; sampling reproduces the model's full distribution, so a word with probability 0.25 is chosen a quarter of the time. Sampling gives variation, greedy gives repetition.

Q7

There is no row for an unseen context. `dist()` returns an empty dict, `predict()` and `sample()` return `None`, and `generate()` stops with status `UNSEEN` instead of crashing (verified: `predict(('banana',))` is `None`, and `generate()` continued from the unseen contexts 'banana' and 'cat park' returns status `UNSEEN` in both modes; output in section 3). A real system would need smoothing or back-off to give unseen events non-zero probability.

Q8

A total of 0.87 means the row is not a valid probability distribution: 13% of the probability mass is missing. Likely causes are a normalisation bug (dividing by the wrong total, or dropping a transition such as one to <END>) or counts modified after normalising. Sampling from that row would be biased. The negative control shows the test catches this.

Q9

Mostly for single-continuation words, but not always. Predictions such as sat -> on, ran -> to and on -> the match what I would expect. The first-order argmax after 'the' is 'cat' only because 'cat' and 'dog' tie at 0.25

and the tie is broken alphabetically; a person has no such preference. Also, the first-order model lets 'the' be followed by 'park', 'mat' or 'rug' regardless of the earlier verb, so it generates 'the cat sat on the park' and even 'the park' as a whole sentence, which a person would not expect. The model reflects the statistics of six sentences, not grammar or meaning: a probability model captures what was frequent in the data, while human expectation uses world knowledge and longer-range structure.

Q10

Sampling produces more variation: 14 distinct sentences in 20 first-order samples, and 4 distinct in 5 samples, whereas greedy returns the identical sentence every time (all five greedy runs per model are listed in section 4). The first-order greedy output is degenerate: 'the cat sat on the cat sat on ...' never reaches <END> and was cut off at the 20-token cap, because the arg max path forms the cycle the $\rightarrow$ cat $\rightarrow$ sat $\rightarrow$ on $\rightarrow$ the. Sampling escapes such cycles because lower-probability choices (e.g. ran, dog) are sometimes taken.

Q11

(1) Graph: first-order has one parent per node ($X_{t-1} \rightarrow X_t$); second-order has two parents (X_{t-2} and X_{t-1} both point to X_t). (2) CPT: rows are indexed by pairs instead of single words, so up to $|V|^2$ rows instead of $|V|$ (121 vs 11 here, counting <START>). (3) Context: two previous tokens instead of one, so 'on the' is distinguished from 'to the'. (4) Data: each row is seen less often, so far more data is needed to estimate the table; 106 of the 121 possible second-order rows have no data at all.

Q12

More context lets the model separate situations that first-order merges: after 'the', first-order can output park even after 'sat on', producing 'the cat sat on the park', while second-order knows 'on the' is followed only by mat or rug. But the CPT has about $|V|^n$ rows for order n , growing exponentially, so with limited data most contexts are never observed and the observed ones have few counts (15 observed contexts here, seen only 1-6 times each). The result is high variance: the second-order model reproduced only training sentences (20 of 20 samples, 6 distinct), i.e. it memorises rather than generalises.

Q13

Approach B states the model, estimation method and generation method, so the LLM output can be checked against a specification; with approach A we would not know what was built. B requires me to understand the representation (a CPT estimated from counts), lets me validate the generated code against it (where counts and probabilities live), lets me test probabilistic invariants (rows sum to 1, unseen contexts handled), and separates the model (the distribution) from the implementation (dicts and a sampling routine): the code can be wrong while the model is right, and only with B can I tell.

Q14

(a) **Factorisation of the joint:** $P(x_1, \dots, x_T)$ is a product of local CPTs, which is why the model can be built from counts row by row. (b) **Independence assumptions:** the graph states that first-order assumes X_t independent of $X_{1:t-2}$ given X_{t-1} , which explains failures such as 'the cat sat on the dog sat on the mat'. (c) **Effect of increasing context:** adding parents enlarges the CPT exponentially (11 $\rightarrow$ 121 rows) and shows the trade-off between expressiveness and estimability. (d) **Testing against the specification:** the sum-to-one property of every CPT row follows from the probabilistic view and gave a direct test. (e) **Principled generation:** ancestral sampling, following the graph from X_1 onwards.

6. Reflection on LLM use

I wrote two prompts for the LLM (Claude Code), reproduced verbatim in Appendix A: a behavioural specification for a first-order model (counts, CPT, arg max, sampling, stop at <END>, no ML libraries), and a request to extend the same program to second order using counts of triples. Before accepting the second-order code I stated what should change: the context becomes a pair of tokens and the <START>

padding doubles. I then read the code and ran it instead of trusting it. Points I inspected, each confirmed by running the program:

- **Greedy decoding can cycle.** Reading the generation loop, greedy mode on the first-order model must follow the arg max path the `-> cat -> sat -> on -> the -> ...`, which never reaches `<END>`. The generated code contains a length cap `MAX_LEN = 20` and returns a status (`END / UNSEEN / MAXLEN`). The run confirms that the cap is needed: the first-order greedy sentence ends with status `MAXLEN`. Without the cap the program would hang.

- **Tie-breaking.** $P(\text{cat}|\text{the}) = P(\text{dog}|\text{the}) = 0.25$, so a bare `max(d, key=d.get)` would depend on dictionary insertion order. The code iterates over `sorted(d)`, so arg max is deterministic (here 'cat').

- **Reproducible sampling.** `random.choices` over dict keys also depends on insertion order. Words are sorted before sampling and a seeded `random.Random` instance is used; re-running the program gives byte-identical output.

- **Unseen contexts.** A direct `probs[ctx]` lookup would raise `KeyError` for an unseen word; `dist()` returns an empty dict and generation stops with status `UNSEEN (Q7)`. Normal generation never reaches an unseen context with this dataset, so during review an optional `prefix` argument was added to `generate()` and the script now continues from an unseen context and asserts the `UNSEEN` status.

- **Validation against the specification.** The normalisation test is separate from the model code, and the negative control (a 0.87 row) shows that it can fail. I also compared rows by hand with the training sentences: 'the' is followed by cat 3, dog 3, mat 2, rug 2, park 2 times (12 total), giving `0.25/0.25/0.167/0.167/0.167`, which is what the program reports. The handout's 3/5 example does not match this dataset.

Appendix A: Prompts used with the LLM

Both prompts were written by me and are reproduced verbatim; long lines are wrapped to fit the page, with continuation lines indented.

Prompt 1: first-order model (Part V)

Write a simple Python implementation of a first-order autoregressive language model.

Training data: a list of tokenised sentences, lower-cased, one token per word. Pad each sentence with a <START> token at the beginning and an <END> token at the end.

The model should:

1. take a list of tokenised sentences as training data;
2. count transitions between consecutive tokens;
3. construct the conditional distribution $P(X_t | X_{t-1}) = C(w_i, w_j) / \sum_k C(w_i, w_k)$;
4. display the probabilities for a specified previous token;
5. predict the most probable next token (break ties alphabetically so the result is deterministic);
6. generate a sentence by repeatedly sampling the next token from $P(X_t | X_{t-1})$, starting from <START>;
7. stop when the <END> token is generated.

Also support a greedy mode that always picks the arg max instead of sampling.

Do not use a machine-learning library or a pretrained language model. Use ordinary Python data structures (dicts of dicts) and the random module, with a seedable random number generator.

Use this training data:

```
the cat sat on the mat
the cat sat on the rug
the dog sat on the mat
the dog ran to the park
the cat ran to the park
the dog sat on the rug
```

Explain in comments where the counts are stored, where the probabilities are computed, and what happens for a word with no observed transitions. Do not assume the code is correct; I will test it.

Prompt 2: second-order modification (Part XII)

Modify the existing first-order autoregressive model into a second-order model.

The model should estimate $P(X_t | X_{t-2}, X_{t-1})$. Represent the model using counts of observed triples and use these counts to construct conditional probability distributions. Pad each sentence with two <START> tokens so that the first word has a context.

Keep the same interface: display the distribution for a given context pair, predict the arg max, sample, greedy mode, and stopping at <END>.

Do not replace the model with a neural network or a pretrained language model.